

Automatic Image Analysis Script

July 1, 2026

Contents

1	Introduction to Automatic Image Analysis	2
1.1	Lecture Aim and the Inverse Problem	2
1.2	From Pixels to Interpretation	3
1.3	Data Types and Interpretation Tasks	5
1.4	Models, Data, and Evaluation	6
1.5	Topic Roadmap	8
2	Neural Networks and Backpropagation	8
2.1	From Perceptrons to Multilayer Networks	9
2.2	Losses, Optimization, and the Need for Gradients	9
2.3	Backpropagation	10
2.4	Automatic Differentiation and Practical Training	11
3	Visual Object Models	11
3.1	Detecting Lines: The Hough Transform	12
3.2	Detecting Arbitrary Shapes: The Generalized Hough Transform	16
3.3	Where to Look: Salient Regions	17
3.4	What to Record: Local Features and Shape Descriptors	19
3.5	Putting It Together: The Implicit Shape Model	20
A	Figures: Visual Object Models	21

1 Introduction to Automatic Image Analysis

Automatic image analysis starts from the central difficulty of vision, then moves through the main processing levels, the recurring goals of the field, and the practical question of how to evaluate whether a vision system is actually reliable.

1.1 Lecture Aim and the Inverse Problem

Automatic image analysis is the problem of turning measurements into explanations. A digital image is easy to store: it is a grid of values. It is much harder to understand: we want to infer objects, surfaces, motion, geometry, lighting, and meaning from those values.

Computer vision is the computational study of how useful descriptions of the visual world can be recovered from images and video. The description may be semantic, such as “there is a pedestrian in front of the car”; geometric, such as “this surface is two meters away”; or operational, such as “reject this part because it contains a defect”. This definition is deliberately broader than object recognition. A vision system may label an image, reconstruct a three-dimensional scene, align several images, estimate motion, improve an image for a human observer, or supply measurements for a larger decision-making system.

Computer vision is therefore not just image enhancement. It is inference under uncertainty. The image is evidence; the scene that produced it is hidden.

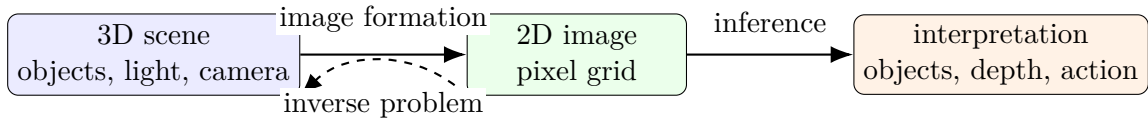


Figure 1: Computer vision tries to invert the image formation process: pixels are observed, but the scene explanation is hidden.

Human vision feels immediate, but this ease is misleading. A camera records a two-dimensional projection of a three-dimensional world. During that projection, many causes are mixed together: object shape, material, illumination, viewpoint, occlusion, sensor noise, and motion. Different scenes can produce very similar images, so there is usually no unique answer that can be read directly from the pixels.

This makes vision an inverse problem. The forward direction is comparatively clear: if we know the scene, the light, the camera, and the materials, we can model how an image is formed. The inverse direction asks us to go backward from image measurements to a plausible explanation of the world. That backward step requires assumptions, models, data, and algorithms.

A compact way to write the forward model is

$$I(x, y) = f(\Pi(X, Y, Z), L, M, C) + \varepsilon, \quad (1)$$

where $I(x, y)$ is the measured pixel value, Π is the camera projection from 3D to 2D, L describes illumination, M material and surface properties, C camera response, and ε sensor noise. Vision asks for some hidden scene state S from the image I :

$$\hat{S} = \arg \max_S p(S | I) = \arg \max_S p(I | S) p(S). \quad (2)$$

The likelihood $p(I | S)$ says which images a scene would probably produce. The prior $p(S)$ encodes what kinds of scenes are plausible before seeing the image.

The gap between the pixel array and the human interpretation is the essential problem. A person immediately infers objects, depth, material, intent, and context, while the measured image itself only contains sampled intensities or colors.

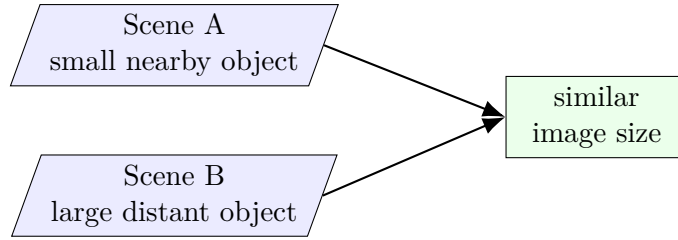


Figure 2: Ambiguity is unavoidable: different scenes can lead to similar image measurements. Priors and additional evidence help choose an explanation.

1.2 From Pixels to Interpretation

Automatic image analysis can be organized as a progression from low-level to high-level tasks.

- Low-level processing improves or transforms image measurements, for example acquisition, sampling, denoising, filtering, contrast changes, and edge extraction. Its typical mapping is image to image, or scene to image when the acquisition process is included.
- Mid-level analysis groups measurements into more meaningful structures, such as regions, contours, features, segments, motion fields, or depth estimates. It is often the bridge from image measurements to feature descriptions.
- High-level interpretation assigns semantic meaning, such as recognizing objects, estimating scene layout, detecting abnormalities, or tracking actions. Its typical mapping is from features or image evidence to object models, object detections, categories, or decisions.

These levels are useful for teaching, but real systems often mix them. A modern recognition model may learn low-level filters and high-level categories inside one network. A classical pipeline may use carefully designed features followed by statistical inference. In both cases, the goal is the same: reduce the ambiguity of image data until a useful decision or description becomes possible.

Level	Typical mapping	Typical question	Examples
Low-level	scene/image → image	What local measurements are reliable?	denoising, gradients, edges
Mid-level	image → features	Which measurements belong together?	regions, contours, segments, motion
High-level	features/image → objects	What does the image mean?	objects, scene layout, actions

Table 1: A teaching taxonomy for image-analysis tasks.

The same hierarchy can also be read as a contrast between bottom-up and top-down processing. A bottom-up system starts with the image data and builds larger hypotheses from local evidence: pixels become edges, edges become contours, contours become regions, and regions become object candidates. This is data-driven and attractive because the image itself constrains the result. Its weakness is that local evidence is often ambiguous or missing.

Top-down processing starts from a model, context, task goal, or prior expectation and asks how the image should be interpreted under that hypothesis. This is knowledge-driven: a rough object model can guide the search for parts, a scene context can make one interpretation more plausible than another, and a classifier can project learned category knowledge onto the image. Top-down processing is powerful because it uses what is already known, but it can also produce confident misinterpretations when the prior is wrong. Practical systems therefore often use a

mixed mode: bottom-up evidence proposes or constrains hypotheses, while top-down models select, refine, or reject them.

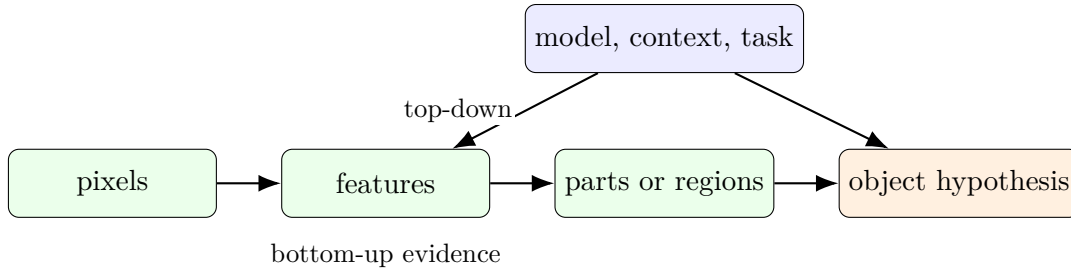


Figure 3: Bottom-up processing builds interpretations from image evidence. Top-down processing uses models and context to guide or correct the interpretation.

Computer vision can be organized around three recurring goals: recognition, reconstruction, and reorganization.

Recognition. Recognition asks what is present and where it is. Examples include classification, detection, segmentation, reading text, identifying faces, or finding defects in industrial inspection.

Reconstruction. Reconstruction asks what spatial or physical structure produced the image. Examples include estimating depth, camera motion, three-dimensional shape, or the geometry of a scene from one or more images.

Reorganization. Reorganization changes the image representation so that later analysis becomes easier or the image becomes more useful to a person. Examples include registration, stitching, stabilization, denoising, super-resolution, and computational photography.

These goals are connected. Recognition benefits from good representations; reconstruction needs reliable correspondences; reorganization often depends on understanding motion, geometry, or image formation.

Area	Main transformation	Typical examples
Digital image processing	image $\rightarrow$ image	enhancement, restoration, filtering, compression
Photogrammetry and geometry	image(s) $\rightarrow$ 3D model	camera calibration, depth, surface reconstruction, mapping
Automatic image analysis	image $\rightarrow$ object model or detection	feature extraction, segmentation, object categorization, recognition

Table 2: Automatic image analysis is closest to the recognition branch of computer vision: it studies how image evidence becomes object-level descriptions, while still using processing and geometric ideas when they support that goal.

Another useful taxonomy separates the vocabulary of the field by the kind of question being asked. Some terms describe the *input* and its formation, some describe *representations* computed from the image, some describe *tasks*, and some describe the *knowledge* used to resolve ambiguity. Keeping these groups separate helps avoid a common beginner confusion: a feature, a model, a task, and an application are not the same kind of object.

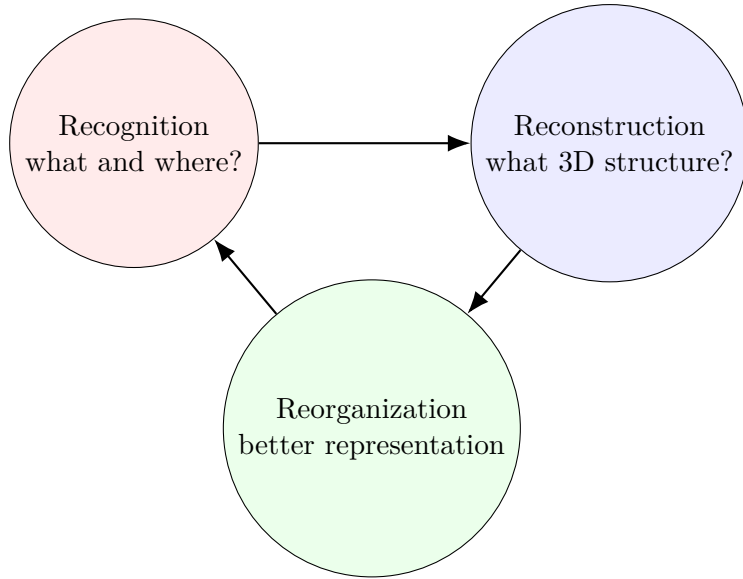


Figure 4: Recognition, reconstruction, and reorganization are not isolated topics; each goal supplies information that can support the others.

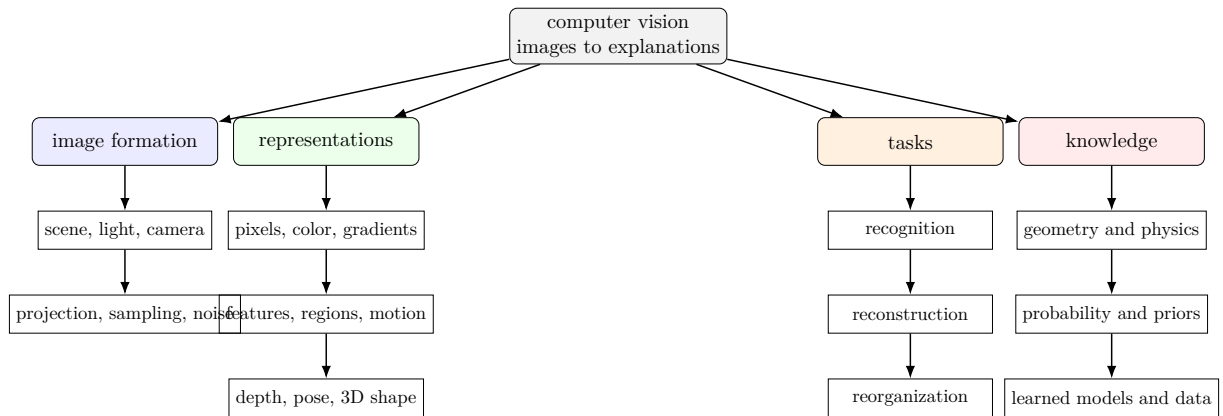


Figure 5: A compact taxonomy of introductory computer-vision terms. The field connects image formation, intermediate representations, task goals, and prior knowledge.

1.3 Data Types and Interpretation Tasks

Before listing concrete tasks, it helps to be precise about what kind of data an image is. We distinguish *structured* from *unstructured* data by whether the arrangement of the values carries meaning. A list of attributes such as (70 kg, 170 cm, 32 years) is unstructured: the entries can be permuted without changing what is described, as long as we remember which field is which. An image is structured, because the spatial layout of the pixels *is* the information. Shuffling the pixels destroys the content even though the multiset of intensity values is unchanged; sound and text are structured for the same reason. This is exactly why image analysis cannot treat a picture as an unordered bag of intensities: position and neighborhood are part of the signal, and the methods in later sections are built to exploit them.

High-level interpretation is not one task but a family of related questions, and it is worth naming them precisely because they differ in what the output looks like. *Image classification* assigns the whole image, or a given region, to one of a fixed set of categories, without saying where anything is. *Object detection* adds localization: it returns a set of objects, each with a position, usually a bounding box, together with a category, so it answers “what” and “where” at once. *Semantic segmentation* pushes localization to the pixel level, assigning a class label

to every pixel and thereby partitioning the image into semantically uniform regions, though without separating individual instances of the same class. *Object recognition* is the broader goal of identifying which objects are present; in its narrow sense it means recognizing a specific identity or instance, such as a particular face or landmark, and in its broad sense it is used as an umbrella over classification and detection. Beyond these, interpretation also includes pose estimation, which recovers the spatial configuration of an object, action recognition, which labels an activity in a video, and attribute classification, which describes properties rather than category.

Task	Output	Question it answers
Image classification	one category for the whole image or region	what is in this image?
Object detection	set of located objects, e.g. boxes plus labels	what is present and where?
Semantic segmentation	a class label for every pixel	which class does each pixel belong to?
Object recognition	identity or category of the objects present	what object is this?
Pose estimation	spatial configuration, e.g. key-points or orientation	how is the object arranged?
Action recognition	activity label for a clip	what is happening over time?

Table 3: High-level interpretation tasks differ mainly in how much localization they require, from a single image-level label up to a decision at every pixel.

These tasks share the same underlying processing chain. Image acquisition and preprocessing produce reliable low-level measurements, mid-level analysis groups them into features and segments, and the high-level stage turns those into the task-specific output above. The choice of task fixes the form of the answer, but the bottom-up evidence and top-down knowledge feeding into it are the same.

1.4 Models, Data, and Evaluation

Because images are ambiguous, vision systems need prior knowledge. Sometimes that knowledge is explicit: camera geometry, physical image formation, smooth surfaces, rigid motion, or probabilistic noise models. Sometimes it is learned from data: neural networks discover features and decision rules from many examples.

Classical computer vision often emphasizes explicit models and engineered features. Modern deep learning often emphasizes learned representations and large datasets. The practical lesson is not that one replaces the other completely. Robust systems usually combine several kinds of knowledge: scientific models for image formation, statistical models for uncertainty, engineering choices for reliability, and data-driven learning for complex patterns that are hard to specify by hand.

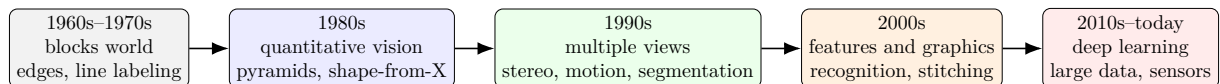


Figure 6: A brief historical timeline. The emphasis moved from hand-built geometric reasoning, through quantitative reconstruction and engineered features, toward large-scale learned representations and integrated sensing.

The timeline is not a story of one idea simply replacing another. Edge operators, camera geometry, variational estimation, probabilistic models, feature matching, and neural networks all remain

useful. What changes over time is the balance between explicit modeling, available computation, available training data, and the kinds of applications that can be deployed reliably.

Source of knowledge	What it contributes	Risk if missing
Physical model	Camera geometry, illumination, projection	Results may violate how images are formed
Statistical model	Noise, uncertainty, priors, inference	The system may overtrust ambiguous measurements
Engineering design	Robustness, speed, failure handling	The method may work only in demonstrations
Data-driven learning	Features and decision rules from examples	Complex visual variation may be underspecified

Table 4: A practical vision system usually combines several kinds of knowledge.

Computer vision is now a basic technology in many domains. It is more useful to name the application categories than to memorize a long list of products: consumer imaging and media, medicine and life sciences, industry and logistics, robotics and autonomous systems, remote sensing and mapping, document analysis, security and biometrics, and scientific measurement. The tasks differ, but the same conceptual questions recur: what image evidence is available, what assumptions make the interpretation possible, and what failure cases matter in the application context?

Application category	Typical vision output	Main reliability concern
Consumer and media	enhanced images, alignment, tracking	visual quality and robustness across devices
Medicine and life sciences	measurements, segmentations, suspected findings	uncertainty, bias, and clinical validation
Industry and logistics	defects, codes, counts, object poses	speed, repeatability, and rare failures
Robotics and autonomy	localization, obstacles, maps, actions	safety under changing environments
Remote sensing and mapping	land cover, 3D structure, change detection	scale, resolution, and registration accuracy
Documents and biometrics	text, identity cues, authenticity signals	privacy, fairness, and adversarial misuse

Table 5: Categorical application areas are easier to study than isolated product examples because they expose recurring reliability questions.

These applications also show why evaluation matters. A result that looks plausible on one example is not enough. Vision algorithms must be tested against noise, changing illumination, clutter, unusual viewpoints, occlusion, and data that differ from the training or development examples.

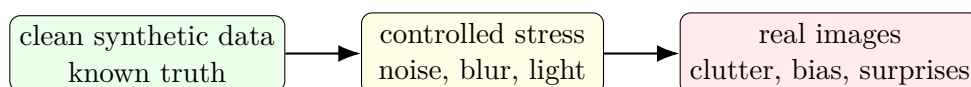


Figure 7: Evaluation should move from controlled correctness toward real-world robustness.

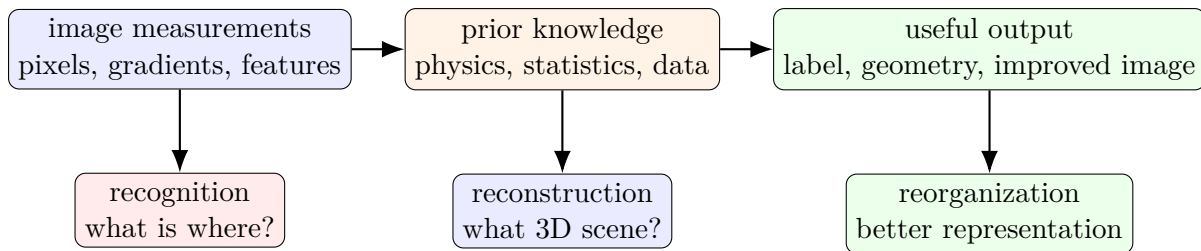


Figure 8: A compact summary of the introductory framing: measurements become useful outputs only after they are combined with assumptions, models, and data.

1.5 Topic Roadmap

Automatic image analysis is narrower than the whole field of computer vision. It uses image processing tools and needs some geometric and probabilistic reasoning, but the main line is automatic interpretation: how images are transformed into features, segments, object models, detections, categories, and evaluated decisions.

The foundations are image representation, filtering, features, transformations, and the basic language of image formation. These topics supply the bottom-up evidence used later. Segmentation, clustering, and feature extraction provide mid-level descriptions that make object-level reasoning possible. Learning-based methods, especially neural networks, show how useful representations and classifiers can be estimated from data. Object models, probabilistic estimation, and higher-level recognition tasks connect these pieces; reconstruction appears where it helps explain image formation, geometry, or the limits of recognition.

Three questions recur throughout automatic image analysis:

1. What information is actually present in the image measurements?
2. What assumptions are needed to infer the hidden scene or decision?
3. How can we test whether the algorithm works beyond the example in front of us?

Automatic image analysis starts with pixels but does not end there. Its real task is to connect images to explanations: what is in the scene, how it is arranged, how it changes, and what action or decision should follow.

2 Neural Networks and Backpropagation

Neural networks are one way to replace manually designed decision rules by learned functions. For image analysis, the input may be an image, a patch, or a feature vector, and the output may be a class score, a probability, a segmentation label, or another task-specific prediction. The central idea is simple: compose many parameterized transformations and fit their parameters from examples.

Szeliski's treatment of deep learning, especially the part on deep neural networks and backpropagation, is a useful companion for this topic. The focus here is narrower: understand the computational structure of a feed-forward network and why backpropagation is the efficient way to compute the gradients needed for training.

2.1 From Perceptrons to Multilayer Networks

A perceptron is a linear classifier. Given an input vector x , weights w , and bias b , it predicts a binary label by thresholding an affine score:

$$f(x) = \begin{cases} 1, & w^\top x + b > 0, \\ -1, & \text{otherwise.} \end{cases} \quad (3)$$

The decision boundary is the hyperplane $w^\top x + b = 0$. This makes the perceptron easy to interpret, but also limited: if the classes cannot be separated by a linear boundary, the standard perceptron cannot solve the problem. The XOR problem is the usual minimal example.

During perceptron training, an incorrectly predicted example changes the weights in the direction of the input:

$$w_i^{t+1} = w_i^t + \eta (y - f(x)) x_i, \quad (4)$$

where η is the learning rate. The rule is useful historically because it already shows the pattern of learning from prediction error, but modern neural networks need a smoother and more general gradient-based view.

A multilayer perceptron (MLP) overcomes the purely linear limitation by stacking several affine maps and nonlinear activation functions. For layer l ,

$$h^l = W^l x^l + b^l, \quad y^l = \phi(h^l), \quad x^{l+1} = y^l. \quad (5)$$

Without the nonlinearity ϕ , stacking layers would still collapse into one linear transformation. The nonlinearities make it possible to build complex decision boundaries from many simple units.

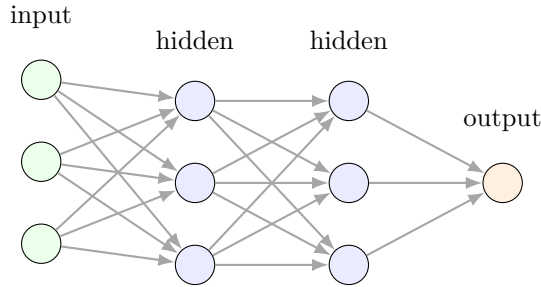


Figure 9: A feed-forward network composes affine transformations and nonlinearities. Training changes the weights on the connections.

Beyond being nonlinear, a useful transfer function is typically monotonic and smooth, so that it has a well-defined derivative everywhere; the classical logistic sigmoid and hyperbolic tangent are additionally bounded, which keeps activations in a fixed range. Common activation functions include the logistic sigmoid, hyperbolic tangent, and rectified linear unit:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z) (1 - \sigma(z)), \quad (6)$$

$$\tanh'(z) = 1 - \tanh^2(z), \quad \text{ReLU}(z) = \max(0, z). \quad (7)$$

For ReLU, the derivative is 1 for positive inputs and 0 for negative inputs; at zero one chooses a convenient subgradient in implementations. This simple derivative is one reason ReLU-style activations are computationally attractive.

2.2 Losses, Optimization, and the Need for Gradients

Training starts with examples (x_i, y_i) and a network f_θ with parameters θ , meaning all weights and biases. A loss function measures how bad a prediction is. Over a dataset, the empirical

objective is

$$L(\theta) = \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i). \quad (8)$$

For regression, ℓ may be a squared error. For classification, it is often cross-entropy applied to class scores or probabilities. The exact loss depends on the task, but the training problem has the same form: find parameters that make the loss small.

Stochastic gradient descent updates parameters using a small batch $\mathcal{B}_t$ instead of the full dataset:

$$\theta_{t+1} = \theta_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i(\theta_t)}{\partial \theta}, \quad (9)$$

where α is the learning rate. The practical difficulty is not writing this update down; it is computing the partial derivatives for millions of parameters quickly and without duplicating work. Backpropagation solves exactly that problem.

Component	Role in the network	Training question
Weights and biases	Store the adjustable model parameters	How should each parameter change to reduce the loss?
Activations	Introduce nonlinear transformations between layers	Which units are active and how do they transmit gradients?
Loss function	Converts predictions and targets into one scalar	What does “better” mean for this task?
Optimizer	Turns gradients into parameter updates	How large and in which direction should the next step be?

Table 6: Training a neural network requires both a model architecture and a procedure for turning prediction errors into parameter updates.

2.3 Backpropagation

Backpropagation is an efficient application of the chain rule to a computational graph. The forward pass computes the network output and stores intermediate quantities. The backward pass starts from the derivative of the loss with respect to the output and propagates sensitivity information back through the same operations in reverse order.

It is helpful to name the quantities at one layer before writing the gradient. Let h_k be the activation vector that enters layer k . The trainable parameters of this layer are a weight matrix Ω_k and a bias vector β_k . The affine part of the layer first forms the pre-activation vector

$$f_k = \beta_k + \Omega_k h_k, \quad (10)$$

and a nonlinearity would then produce the next activation, for example $h_{k+1} = \phi(f_k)$. With this notation, the gradient with respect to the weight matrix has the structure

$$\frac{\partial \ell_i}{\partial \Omega_k} = \frac{\partial \ell_i}{\partial f_k} h_k^\top. \quad (11)$$

This formula captures a useful intuition from the lecture: the effect of a weight depends both on the upstream error signal and on the activation feeding into that weight. If the previous activation is doubled, the same small weight change has twice the local effect.

For ReLU hidden units, the backward recurrence includes the pointwise derivative of the activation:

$$\frac{\partial \ell_i}{\partial f_{k-1}} = \mathbb{I}[f_{k-1} > 0] \odot \left(\Omega_k^\top \frac{\partial \ell_i}{\partial f_k} \right), \quad (12)$$

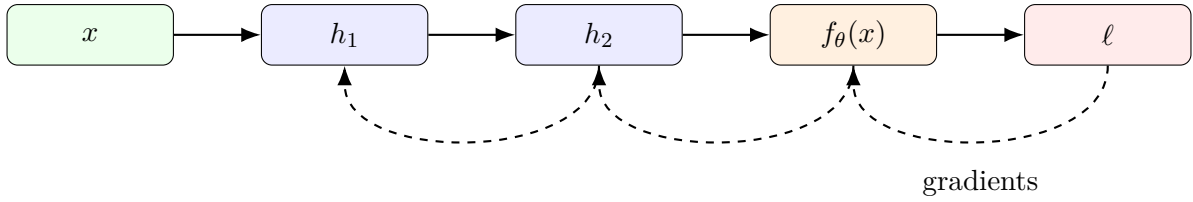


Figure 10: The forward pass evaluates the network from left to right. The backward pass propagates derivatives from the loss back through the stored intermediate quantities.

where $\odot$ denotes elementwise multiplication. The indicator gates the gradient: inactive ReLU units do not pass local gradient information backward.

The important point is not to memorize every indexed expression, but to know what must be stored and what is reused. The activations from the forward pass are needed during the backward pass. This makes backpropagation efficient, but also memory-hungry for large networks.

2.4 Automatic Differentiation and Practical Training

Modern deep-learning libraries implement backpropagation through automatic differentiation. A model is represented as an acyclic graph of operations. Each operation knows how to compute its output in the forward direction and how to propagate derivatives in the reverse direction. The user specifies the model, the loss, and the optimizer; the framework records the graph and applies the chain rule.

A typical training loop has the following structure:

1. initialize the network parameters;
2. choose a loss function and optimizer;
3. load a batch of examples;
4. run the forward pass and compute the loss;
5. run the backward pass to compute gradients;
6. update the parameters and repeat.

This is the conceptual core behind PyTorch-style code: zero old gradients, evaluate the model, call backward on the loss, and let the optimizer step.

Several practical issues remain outside the pure gradient formula. Poor initialization can make training slow or unstable. Too large a learning rate can overshoot; too small a learning rate can make progress painfully slow. Deep models may fit the training data while generalizing poorly. In the usual bias–variance language, underfitting corresponds to high bias, while overfitting corresponds to high variance. Regularization, more data, data augmentation, dropout, weight decay, batch normalization, and early stopping are common ways to improve generalization.

3 Visual Object Models

An object model is a computational answer to a deceptively simple question: what does an object look like, expressed in a form an algorithm can search for? A person has no trouble saying that a coffee mug has a rounded body and a handle, but a detector needs something concrete: a parametric shape, a set of edge displacements, a collection of characteristic image patches, or a learned feature map. The shape of that representation decides which transformations the detector can tolerate (translation, scale, rotation, partial occlusion) and how the search through the image is organized.

Symptom	Meaning	Typical response
High training error	The model or optimization setup is too weak	increase capacity, train longer, adjust learning rate
Low training error but high validation error	The model overfits the training set	use regularization, more data, or early stopping
Unstable loss	Updates are too aggressive or scales are poor	lower learning rate, improve initialization, normalize inputs

Table 7: Most neural-network training failures can be diagnosed by comparing training behavior with validation behavior.

This section develops object models through a single recurring idea that ties its parts together:

Local image evidence accumulates into global object hypotheses by voting.

Each fragment of the image (an edge, a gradient, a patch) is weak and ambiguous on its own, but it is consistent with only a limited set of global explanations. If every fragment casts a vote for the explanations it supports, the correct hypothesis collects many votes and stands out as a peak in a parameter space, while accidental coincidences stay near the noise floor. We follow this idea through increasingly flexible models: edges first vote for *lines* (Hough transform), then for the center of a *known shape* (generalized Hough transform), and finally learned appearance *parts* vote for the center of an *object category* (implicit shape model). Saliency and descriptors sit in between, deciding *which* local evidence is worth trusting in the first place.

3.1 Detecting Lines: The Hough Transform

The simplest non-trivial object model is a straight line, possibly broken by noise or occlusion into many short edge fragments. The difficulty is that the fragments are scattered across the image, and we do not know in advance how many lines there are or where they lie. Testing every possible line against every group of pixels would be hopeless. The Hough transform sidesteps this by changing coordinates: instead of grouping pixels in the image, it lets every pixel vote in the space of all possible *lines*, and then simply looks for the lines that received many votes.

To make “the space of all lines” concrete we need a way to label each line by a few numbers. The familiar slope–intercept form $y = mx + b$ is a poor choice: a vertical line has infinite slope, so m and b cannot represent it. The *Hesse normal form* sidesteps this by describing a line through the one point it can never help but single out — the point on the line closest to the origin.

Every line has a unique closest point to the origin: the foot of the perpendicular dropped from the origin onto the line. We can reach that foot by starting at the origin and walking a fixed distance $r \geq 0$ in a single direction. Call θ the angle of that direction, so the unit vector pointing from the origin straight at the line is

$$\hat{\mathbf{n}} = (\cos \theta, \sin \theta), \quad (13)$$

so the foot of the perpendicular is the point

$$P_0 = r \hat{\mathbf{n}} = (r \cos \theta, r \sin \theta). \quad (14)$$

The vector from the origin to P_0 is r times $\hat{\mathbf{n}}$, so it is *normal* (perpendicular) to the line, because the shortest path from a point to a line always meets it at a right angle. The pair (r, θ) then pins the line down completely: θ says which way the line faces and r says how far out it lies.

To turn this picture into an equation, take any point $P = (x, y)$ on the line and look at the vector $P - P_0$ that runs from the foot to it. Because both P and P_0 lie on the line, this vector

points *along* the line; and the line is perpendicular to the origin-to-foot vector P_0 . Orthogonality of $P - P_0$ and P_0 means their dot product vanishes,

$$(P - P_0) \cdot P_0 = 0 \quad \Longleftrightarrow \quad P \cdot P_0 = P_0 \cdot P_0. \quad (15)$$

Both sides are now immediate. On the left, $P \cdot P_0 = x(r \cos \theta) + y(r \sin \theta) = r(x \cos \theta + y \sin \theta)$; on the right, $P_0 \cdot P_0 = r^2(\cos^2 \theta + \sin^2 \theta) = r^2$, using $\cos^2 \theta + \sin^2 \theta = 1$. Equating the two gives $r(x \cos \theta + y \sin \theta) = r^2$, and cancelling one factor of r (every line we care about has $r > 0$) leaves the Hesse normal form of the line,

$$x \cos \theta + y \sin \theta = r. \quad (16)$$

Figure 11 shows the whole construction. What looked like an arbitrary combination of a cosine and a sine is really one clean geometric statement: from every point P on the line, the step $P - P_0$ to the foot leaves at a right angle to the origin-to-foot vector P_0 .

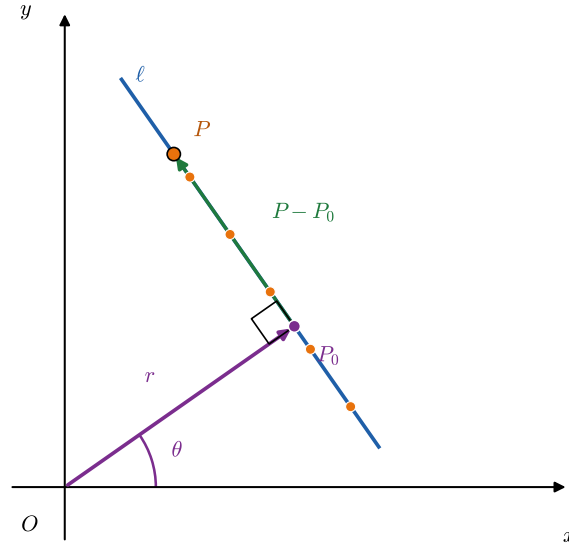


Figure 11: The Hesse normal form of a line. The foot of the perpendicular from the origin O is $P_0 = (r \cos \theta, r \sin \theta)$, reached by walking a distance r at angle θ . For any point P on the line ℓ , the vector $P - P_0$ runs along the line and is therefore orthogonal to P_0 ; the condition $(P - P_0) \cdot P_0 = 0$ is exactly $x \cos \theta + y \sin \theta = r$, which every collinear edge point (orange) satisfies with the same (r, θ) .

Here x and y are pixel coordinates, $r \geq 0$ is measured in pixels, and θ ranges over $[0, \pi)$. A quick sanity check fixes the meaning of the two numbers. A horizontal line at height $y = c$ has a vertical perpendicular, so $\theta = 90^\circ$ and Equation (16) reduces to $r = c$; a vertical line $x = d$ has $\theta = 0^\circ$ and $r = d$; a line passing through the origin has $r = 0$ at every one of its points, whatever its angle. No line, not even a vertical one, makes the parameters blow up. That is exactly the robustness the slope–intercept form lacks.

The decisive property of Equation (16) is an *invariance*: every pixel that lies on one particular line produces the *same* pair (r, θ) , because r and θ describe the line, not the individual point. A whole line in the image therefore collapses to a single point in the (r, θ) *parameter space*. Now read the same equation the other way around. Fix a single image point (x_0, y_0) and let θ vary; the equation turns into a function of θ alone,

$$r(\theta) = x_0 \cos \theta + y_0 \sin \theta, \quad (17)$$

a fixed weighted sum of a cosine and a sine. Any such combination collapses to a single shifted sine of the same frequency, and it is worth seeing exactly why, because the shift is where the phase φ comes from. Write down the shifted sine we are aiming for and expand it with the angle-addition identity $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$:

$$R \sin(\theta + \varphi) = \underbrace{R \cos \varphi}_{\text{weight of } \sin \theta} \sin \theta + \underbrace{R \sin \varphi}_{\text{weight of } \cos \theta} \cos \theta. \quad (18)$$

For this to equal $x_0 \cos \theta + y_0 \sin \theta$ for *every* θ , the two expressions must have the same weight on $\cos \theta$ and the same weight on $\sin \theta$ separately. Matching them term by term gives

$$x_0 = R \sin \varphi, \quad y_0 = R \cos \varphi. \quad (19)$$

These two relations determine R and φ from the point alone. Squaring and adding them cancels φ (via $\sin^2 + \cos^2 = 1$) and leaves the amplitude,

$$R = \sqrt{x_0^2 + y_0^2}, \quad (20)$$

which is simply the point's distance from the origin; dividing them instead cancels R and fixes the phase,

$$\varphi = \text{atan2}(x_0, y_0). \quad (21)$$

Here atan2 is the two-argument arctangent, which recovers the angle in the correct quadrant from the signed pair (x_0, y_0) . So φ is not a new free parameter that appears from nowhere: it is the phase *forced* on the sinusoid by where the point sits, namely the angle of (x_0, y_0) measured from the y -axis (that is what Equation (19) says, since $(x_0, y_0) = (R \sin \varphi, R \cos \varphi)$). Reassembling the pieces,

$$r(\theta) = x_0 \cos \theta + y_0 \sin \theta = R \sin(\theta + \varphi), \quad (22)$$

which is a sinusoid in the (r, θ) plane whose amplitude R is just the point's distance from the origin and whose phase φ encodes its direction. The sinusoid enumerates *all* the lines that pass through that one point. Two pixels that happen to lie on a common line have sinusoids that cross at the (r, θ) of that shared line; many collinear pixels produce many sinusoids crossing at a single point. That crossing is the peak we are looking for. Figure 13 shows both readings side by side: collinear points in the image (one point, one sinusoid) versus the single bright accumulator cell where the sinusoids meet.

So far the “points” that vote have been abstract, but on a real image they are anything but. We do not start with a tidy list of points; we start with a grid of brightness values, and the first job is to decide which pixels are even worth a vote. That is what *edge detection* does: it flags the pixels that sit on a strong local change in brightness — the outlines where a dark region meets a light one — and leaves the smooth interior, which carries no boundary information, silent. These high-contrast *edge pixels* are the evidence the transform actually works with. A straight edge in the scene appears as a run of collinear edge pixels, and it is exactly their sinusoids that all pass through the one (r, θ) of that edge. A peak in parameter space is therefore not a geometric coincidence but a direct tally of how many real edge pixels line up: the more pixels that genuinely share a line, the taller the crossing they build, and the further it stands above the scatter left by unrelated pixels.

This is also where the name *Radon transform* comes from. We cannot solve for (r, θ) directly, because a single point lies on infinitely many lines. So we discretize the parameter space into a grid of cells, an *accumulator* $H_L(r, \theta)$ initialized to zero, and let each edge pixel add one vote to every cell on its sinusoid: for the pixel, sweep θ over the grid, compute the matching r from Equation (16), and increment $H_L(r, \theta)$. After all pixels have voted, a real line shows up as a pronounced maximum, while cells that received only scattered, accidental votes stay low. Detecting lines has become peak-finding in a 2D array.

It helps to picture H_L literally as an image and to set it next to the picture it came from, which is what Figure 12 does: the edge map on the left, the accumulator on the right. The horizontal axis is the angle θ and the vertical axis is the distance r — here allowed to take either sign, so that θ needs to run only over a half-turn instead of a full one. The plane is chopped into a grid of *bins*, and the number of bins along each axis sets the resolution of the search: too coarse and neighbouring lines merge into one cell, too fine and the votes for a single line scatter across several. The grid starts completely black, at zero. Each edge pixel then paints its sinusoid of votes across it, brightening exactly one bin in every θ column, so on its own a pixel leaves only a faint curve. Where many collinear pixels overlap, their sinusoids pile into the *same* bin, which flares into a concentrated bright hotspot, while the accidental votes of unrelated pixels stay dim and diffuse. Detecting lines is then nothing more than thresholding this image for its brightest cells and reading off their (r, θ) .

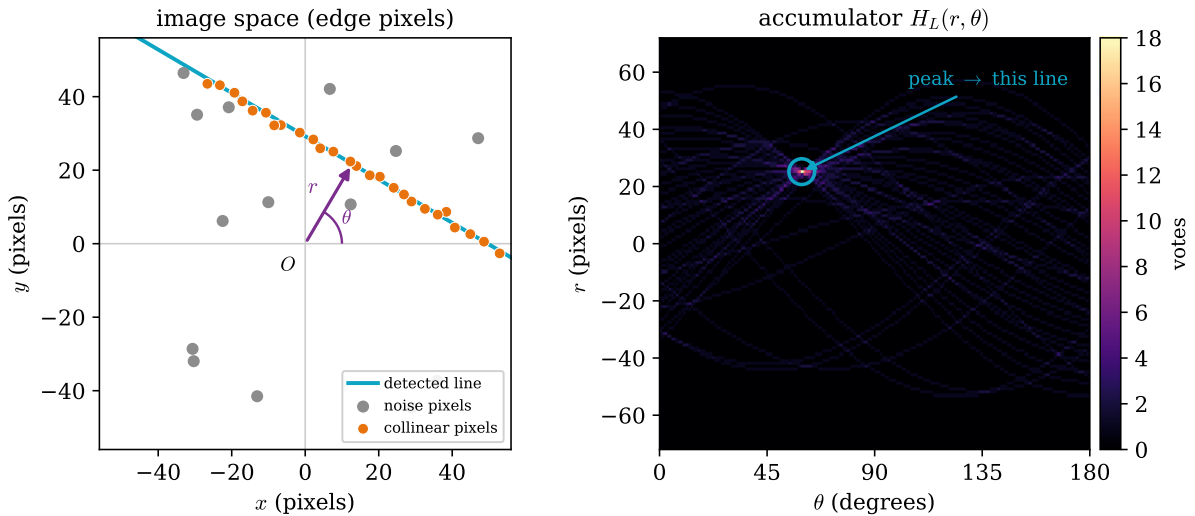


Figure 12: Image space and parameter space side by side, for a synthetic edge map of 26 collinear pixels (orange) plus 14 scattered ones (grey). *Left*: the edge pixels, with the detected line and its perpendicular parameters r and θ measured from the origin O . *Right*: the accumulator $H_L(r, \theta)$ drawn as an image. Every edge pixel casts one vote per θ column, tracing a sinusoid across the grid; the collinear pixels’ sinusoids all cross in the same bin, which lights up as the bright peak (circled) at $(\theta, r) \approx (60^\circ, 25 \text{ px})$, whereas the noise pixels leave only faint, uncrossed curves. That peak *is* the line on the left: line detection reduces to thresholding this grid for its hottest cells and reading off their (r, θ) . Brightness is the number of votes a bin has received.

Using gradients. Making every edge pixel sweep over all directions θ is wasteful, because it throws away information the edge detector already handed us for free. Finding an edge means measuring, at each pixel, the direction in which the brightness changes fastest: the direction of *steepest contrast*, pointing straight across the boundary from the dark side to the light side. That direction is the image *gradient*. Now the geometric twist that makes it so useful — and it is easy to get backwards. The brightness changes fastest as you step *across* an edge and barely at all as you walk *along* it, so the gradient points *perpendicular* to the edge, not along it. The direction of maximum contrast tells us which way the boundary faces; the edge itself runs at a right angle to that. But “perpendicular to the edge” is precisely the *normal* direction of the line through the pixel — the same normal $\hat{n}$ whose angle we named θ in the Hesse form. The contrast direction therefore hands us θ directly, and with it the single line that can pass through that pixel, so the pixel casts one sharp, well-aimed vote instead of a whole sinusoid of guesses. To make this concrete, write the gradient of the image brightness as the pair $(G_x(x, y), G_y(x, y))$, the rates

of change in the x and y directions (for example the outputs of a Sobel filter). The gradient direction

$$\theta_G(x, y) = \text{atan2}(G_y(x, y), G_x(x, y)) \quad (23)$$

already fixes the angle θ of the line through that pixel, so there is no need to try every θ : we compute the single matching r from Equation (16) and update one cell instead of a whole sinusoid. (The function atan2 is the two-argument arctangent; unlike $\arctan(G_y/G_x)$ it returns the angle in the correct quadrant and does not divide by zero when $G_x = 0$.) Finally, a sharp, high-contrast edge is stronger evidence for a line than a faint one, so instead of adding a flat $+1$ we weight each vote by the gradient magnitude,

$$H_L(r, \theta) \leftarrow H_L(r, \theta) + \sqrt{G_x(x, y)^2 + G_y(x, y)^2}, \quad (24)$$

where $\sqrt{G_x^2 + G_y^2}$ is the length of the gradient vector, that is, how steeply the brightness changes at the pixel. A line in the image then produces a pronounced maximum in H_L , found by the same simple peak detection as before.

Points versus edges. The contrast between voting with bare points and voting with oriented edges is worth keeping in mind, because it recurs in every later method. A bare point is consistent with infinitely many lines and must vote for all of them (its full sinusoid), which inflates the chance that unrelated points accidentally agree and forces a higher peak threshold. An oriented edge, by contrast, carries a direction, pins down θ through Equation (23), and votes for essentially one line. The lesson generalizes: the more informative each piece of local evidence is, the cleaner the peaks and the fewer the false alarms.

3.2 Detecting Arbitrary Shapes: The Generalized Hough Transform

Lines are special because they have a tidy two-parameter equation. Most objects do not: a wrench, a leaf, or a logo is an arbitrary 2D curve with no closed-form parameterization. The *generalized Hough transform* (GHT) keeps the voting idea but replaces the parametric equation with a stored table that records the shape's geometry explicitly. The detector still asks the same question every edge can answer with a vote, only now about a shape rather than a line: *if you are part of this shape, where is its center?*

To set this up we first pick one *reference point* on the model shape, typically (but not necessarily) its center of mass. This reference point plays the role that (r, θ) played for lines: its position in a test image is what we are voting for. During training we walk along the model contour, and for each edge pixel at position (x, y) we record the *displacement vector* (d_x, d_y) that points from that edge to the reference point, so that

$$(x_{\text{ref}}, y_{\text{ref}}) = (x + d_x, y + d_y). \quad (25)$$

The logic at detection time is the mirror image of Equation (25): if an observed edge in the test image is the same part of the shape, then the center must lie at the edge's own position plus one of the stored displacements, so the edge votes for those locations. Edges that genuinely belong to one instance of the shape all point back to the same spot, which lights up as a peak; unrelated edges scatter their votes harmlessly.

What keeps this efficient is the *R-table*, an indexed list of the displacement vectors. Just as the gradient orientation accelerated line detection, it organizes the table here. We quantize the possible gradient directions into A equal bins (rows). Row T covers gradient orientations near $\theta \approx 2\pi T/A$ for $T \in \{0, 1, \dots, A-1\}$, and it stores every displacement vector that was observed at an edge with roughly that orientation:

$$\text{row } T : \quad (d_x^{T,1}, d_y^{T,1}), (d_x^{T,2}, d_y^{T,2}), \dots, (d_x^{T,N_T}, d_y^{T,N_T}). \quad (26)$$

Concretely, with $A = 8$ bins each row spans 45° ; an edge whose gradient points at 100° falls in the row centered near 90° , and we read off exactly that row’s vectors. The superscripts in Equation (26) just enumerate the N_T different vectors stored in row T — there can be several, which is the point of the next paragraph. At test time, then, the procedure is: read each edge’s gradient orientation, jump to the matching row, and cast a vote at $(x + d_x, y + d_y)$ for every displacement (d_x, d_y) in that row.

Why does one row hold several vectors? Because the local gradient is *ambiguous*: the same gradient direction occurs at several different places on a shape (think of the two opposite sides of a circle, whose outward normals are antiparallel, or the many points along a smooth curve that happen to face the same way). A given orientation is therefore consistent with several possible centers, so a single observed edge generally increments *several* accumulator cells, one per displacement in its row. This is not a defect to be engineered away; it is precisely why voting is needed. Each edge is individually uncertain about where the center is, but only the true center collects support from many edges at once. Figure 14 traces this from the model contour and its displacement vectors through the R-table to the accumulator peak.

Real instances also differ from the stored template in size and orientation. To handle this, the accumulator gains two more axes and becomes four-dimensional, $H(x, y, s, \theta)$, where (x, y) is the candidate center, s a scale factor, and θ a rotation angle. The two phases are then:

Training. Extract the edge pixels of the template; for each, compute its gradient orientation and store the displacement vector to the reference point in the corresponding R-table row (Equation (26)).

Prediction. For each scale s , multiply the stored displacement vectors by s ; for each rotation θ , rotate the vectors by θ and shift the R-table rows around by the same amount (a circular shift, since rotating the shape rotates every edge’s gradient by θ); then, for each edge pixel in the test image, look up its row, cast a vote from each of its displacement vectors, and increment $H(x, y, s, \theta)$. Finally, find the peaks in the four-dimensional array.

The price of this generality is the four-dimensional search; the reward is a detector for an *arbitrary* known shape rather than a single parametric family of lines.

3.3 Where to Look: Salient Regions

The generalized Hough transform still assumes we possess an exact template of the object. That assumption breaks for object *categories*: no two cars, faces, or motorbikes share one outline, so there is no single contour to store in an R-table. Before we can let parts vote for an object, we have to decide which image locations even deserve to be treated as parts. This is the problem of *saliency*: detecting distinctive regions whose local appearance stands out from their surroundings, so that the same regions can be found again on a new, previously unseen instance.

The motivation is partly biological. Saliency is a subjective perceptual quality that makes certain items capture an observer’s attention before others. A standard computational account mirrors early vision: simple features (color, edge orientation, luminance, motion) are measured in parallel across the whole field and at several scales, producing a stack of *feature maps*. Each map is processed to emphasize locations that differ from their neighbors, the maps are combined into a single saliency map, and attention — and, in a living visual system, the eyes and head — is drawn to the most salient locations. The idea we borrow for object models is this notion of “standing out from the surroundings,” made mathematically precise and, importantly, invariant to scale and rotation so that detections repeat across instances.

The Kadir/Brady detector makes “standing out” concrete by equating saliency with *rareness*, measured through Shannon entropy. The reason rareness needs a careful definition is that it is easy to get trivially wrong: if our description of a patch is so specific that every location looks unique, then everything is equally rare; if it is so generic that everything looks alike, nothing is

rare. Entropy of the local intensity distribution sits between these extremes and measures the *complexity* of a patch. For a window of pixels, let $p(k)$ be the fraction of pixels whose grey value equals k , for $k = 0, \dots, 255$; this is just the normalized local histogram. The entropy is

$$H = - \sum_{k=0}^{255} p(k) \log_2 p(k), \quad \text{with the convention } 0 \log_2 0 = 0. \quad (27)$$

Each term $-p(k) \log_2 p(k)$ is the contribution of grey level k ; summing them gives the average number of *bits* needed to encode a pixel drawn from the window, which is also a measure of how uncertain we are about that pixel's value. A couple of concrete windows fix the intuition. If every pixel has the same grey value, one $p(k)$ equals 1 and the rest are 0, so $H = 0$: a flat, predictable patch carries no information. If the window contains two grey values in equal numbers, two terms equal $-\frac{1}{2} \log_2 \frac{1}{2} = \frac{1}{2}$ and $H = 1$ bit. If all 256 levels are equally likely, $p(k) = 1/256$ for each and $H = \log_2 256 = 8$ bits, the maximum. So a *flatter* local histogram means *higher* entropy, and a textured, unpredictable patch is rare in this spatial sense, while a smooth or constant patch is not.

High spatial entropy alone is still not enough, because pure noise also has a flat histogram yet is not distinctive: its statistics barely change as the analysis window grows, so it looks the same at every scale. We therefore add a second requirement, *self-dissimilarity in scale*: a salient region should look different at its own scale than at nearby scales. To express this we analyze each location over a range of window sizes (a Gaussian scale space, or an image pyramid that subsamples in logarithmic steps), and treat entropy as a function of the scale s . Writing $p(I, s, x)$ for the probability density of intensity value I inside a window of scale s centered at position x , the scale-dependent entropy and the scales at which it peaks are

$$H_D(s, x) = - \int p(I, s, x) \log_2 p(I, s, x) dI, \quad s_p = \left\{ s : \frac{\partial H_D}{\partial s} = 0, \frac{\partial^2 H_D}{\partial s^2} < 0 \right\}. \quad (28)$$

Equation (28) is just Equation (27) written for a continuous intensity I and made a function of scale; the condition on s_p is the textbook test for a local maximum (first derivative zero, second derivative negative), so s_p collects the scales where the entropy curve has a *peak*. Each such peak is then reweighted by how strongly the intensity distribution itself changes with scale,

$$W_D(s, x) = s \int \left| \frac{\partial}{\partial s} p(I, s, x) \right| dI, \quad y(s_p, x) = H_D(s_p, x) W_D(s_p, x). \quad (29)$$

The weight W_D is large when the distribution looks markedly different at scale s than at neighboring scales (the $|\partial p / \partial s|$ term), with the leading s making the measure comparable across scales; the final saliency $y(s_p, x)$ is the product, so a location scores highly only when it is both complex (high H_D) and tied to a distinctive scale (high W_D). This is exactly what suppresses featureless noise, whose entropy is high but flat across scale. Figure 15 shows the entropy-versus-scale curve with its peak, alongside the flat-versus-peaked histogram intuition.

In practice the algorithm visits each pixel, computes the local entropy (Equation (27)) over a range of scales, keeps the scales where the entropy peaks (there may be none), and weights those entropies by the inter-scale change of the distribution. Run on a dataset such as side views of motorbikes, and averaged over many images, the detected salient points cluster on repeatable, characteristic object parts — wheels, the engine block, the seat — which is precisely the raw material the next models need.

This is the *classical* stance on saliency: hand-crafted features grounded in information theory, computed without learning, and largely task-independent (data enter only at evaluation time). The *modern* alternative is to learn saliency from data with deep networks, building on the multilayer perceptrons and convolutional networks of Section 2; a convolutional network trades hand-designed rareness for features learned from many labeled examples, at the cost of needing a great deal of such data. We stay with the classical view here because it makes explicit *why* a location is worth voting from.

3.4 What to Record: Local Features and Shape Descriptors

Saliency tells us *where* to look; a descriptor decides *what* to record there, encoding the neighborhood as a vector so that the same part can be matched again on a new image. What makes one descriptor better than another is best understood as a balancing act among four competing demands, summarized in Table 8.

Property	What it means
Invariance	Stays stable under nuisance changes: translation, rotation, scale, and illumination.
Discriminability	Separates similar classes reliably; different parts get different descriptors.
Compactness	Stores enough information in a short vector to keep matching and storage cheap.
Computational cost	Remains feasible to extract and compare on large datasets.

Table 8: The four criteria for a good local feature. Invariance and discriminability pull in opposite directions, and compactness and cost bound how far either can be pushed.

These demands genuinely conflict. Pushing invariance to the limit deliberately discards information — if a descriptor is built to ignore rotation, it can no longer tell two parts apart that differ only by rotation — which eats into discriminability. And richer, more discriminative descriptors are longer and costlier to compute and compare. Every concrete descriptor is one particular compromise among the four.

The *histogram of oriented gradients* (HOG) is the canonical compromise on the shape-and-appearance side, and it is worth seeing how it is built because it reuses the gradients from Equation (24). The region is divided into a grid of small *cells*; within each cell the gradient orientations of the pixels are collected into a histogram (for example over nine orientation bins), each pixel contributing in proportion to its gradient magnitude. Cells are then grouped into overlapping *blocks*, and the histograms in a block are normalized together before being concatenated, over all blocks, into one long feature vector. Two design choices give HOG its character. Summarizing gradient *orientations* rather than raw intensities, together with the block normalization, makes it robust to moderate lighting changes and small deformations. But because the descriptor is laid out as a fixed grid of oriented histograms, rotating the patch permutes the bins and changes the vector: standard HOG is *not* rotation invariant. In return it turns the diffuse evidence of many edges into a single classifier-ready vector, which is why it remains a standard baseline against which learned features are measured.

Two further descriptor families round out the classical toolkit and contrast usefully with HOG. *Fourier descriptors* encode a closed contour by the Fourier coefficients of its boundary curve; translation, rotation, and scaling of the shape act on those coefficients in simple, separable ways (a shift of the constant term, a phase change, an overall magnitude scaling), so after normalization the coefficients become invariant shape descriptors. *Hu moments* combine image moments into a handful of quantities that are likewise invariant to translation, scale, and rotation. For detecting the salient *points* in the first place, the standard keypoint operators (Harris, Förstner, SIFT) provide the geometric counterpart to the information-theoretic salient regions of the previous subsection. The common thread across all of them is the same: a descriptor converts a neighborhood into a compact vector, and closeness between two such vectors is taken to mean that the two patches show the same thing.

3.5 Putting It Together: The Implicit Shape Model

The implicit shape model (ISM) is where the three threads of this section — voting, saliency, and description — finally meet. Its goal is object *categorization* in real scenes: given training images of a category, learn a model from the local appearance of its parts; given a new image, both label and segment the instances. The one-sentence summary is that ISM is a generalized Hough transform whose fixed geometric R-table has been replaced by a *learned appearance codebook*. Where the GHT indexed displacement vectors by an edge’s gradient orientation, ISM indexes them by a patch’s appearance; the voting machinery is otherwise the same.

The **training phase** builds that codebook. Salient points are extracted from the training images (using the salient-region or keypoint detectors above), and each one defines a *patch* at its characteristic scale. The patches are described by their local appearance and then grouped so that visually similar patches become a single *codebook* entry — a prototype part such as “front wheel” or “headlight.” The grouping is done by iterative merging: every patch starts as its own cluster, and at each step the two most similar clusters C_1 and C_2 are merged, provided their average pairwise similarity stays above a threshold t . Similarity between two patches p and q is the normalized greyscale correlation, and the similarity of two clusters averages it over all their members:

$$\text{NGC}(p, q) = \frac{\sum_i (p_i - \bar{p})(q_i - \bar{q})}{\sqrt{\sum_i (p_i - \bar{p})^2 \sum_i (q_i - \bar{q})^2}}, \quad \text{sim}(C_1, C_2) = \sum_{p \in C_1, q \in C_2} \frac{\text{NGC}(p, q)}{|C_1||C_2|}. \quad (30)$$

In $\text{NGC}(p, q)$, the index i runs over the pixels of the patch, p_i and q_i are their grey values, and $\bar{p}, \bar{q}$ are the patch means; subtracting the means removes brightness offsets and the denominator removes contrast scaling, so NGC is a correlation coefficient between -1 and 1 that equals 1 for patches identical up to brightness and contrast and near 0 for unrelated ones. The cluster similarity $\text{sim}(C_1, C_2)$ is then just the average NGC over all cross-pairs, with $|C_i|$ the number of patches in cluster C_i . Stopping the merges at the threshold t keeps clusters compact and groups only genuinely similar patches. Finally, for each codebook entry we record, across all the training images, the offset vectors from every matching patch to the object center — the appearance analogue of the R-table’s displacement vectors, specifying where on the object each prototype part tends to appear.

The **prediction phase** runs the voting. Salient patches are extracted from the test image and matched against the codebook; an entry is *activated* whenever its NGC similarity to a patch exceeds the threshold t . Each activated entry then casts generalized Hough votes for the object center, placing a vote at the patch position plus each of that entry’s stored offsets — exactly the GHT step, but driven by appearance rather than edge orientation. Unlike the discrete accumulator of the classical GHT, ISM votes in a *continuous* space, and the densest cluster of votes is located with mean-shift mode estimation (equivalently, a non-parametric density estimate); that mode is the hypothesized object center. A final *back-projection* step reverses the voting: the patches that supported the winning center are projected back into the image, which both localizes the object within a surrounding region and, through a probabilistic formulation, yields a pixel-level *segmentation* rather than a mere bounding box. Figure 16 lays out the whole pipeline, training above and prediction below.

Because ISM grows directly out of the generalized Hough transform, it helps to see the two side by side; Table 9 lines up what changes and what stays the same.

Two further models slot naturally into the same picture. *Hough forests* replace the codebook-and-matching step with a random forest that maps each patch directly to a distribution of votes, learning the appearance grouping and the offsets together in one trained predictor. *Deformable part models* (DPM) take a complementary route: an object is a coarse root HOG template plus a few higher-resolution part templates attached to it by spring-like deformation costs, the whole thing scored with a latent-variable classifier. Both keep the central bargain of this section — local parts proposing global configurations — while changing how the parts are represented and

Aspect	Generalized Hough transform	Implicit shape model
Model of a part	Fixed geometric R-table	Learned appearance codebook
Local evidence	Edge + gradient orientation	Salient patch + appearance
Indexing	Gradient-orientation row	Codebook match above threshold t
Voting space	Discrete accumulator	Continuous (mean-shift modes)
Output	Center / pose of one known shape	Category detection <i>and</i> segmentation

Table 9: From a single known shape to an object category: ISM keeps the voting machinery of the generalized Hough transform but learns the parts and their spatial distribution from data.

how their votes are combined. The implicit shape model itself is due to Leibe, Leonardis, and Schiele (ECCV’04 workshop; IJCV 2008).

It is worth ending where we began. Every model in this section is one instance of the same mechanism, differing only in what counts as local evidence and how much flexibility the global hypothesis is allowed; Table 10 lines them up along that single axis.

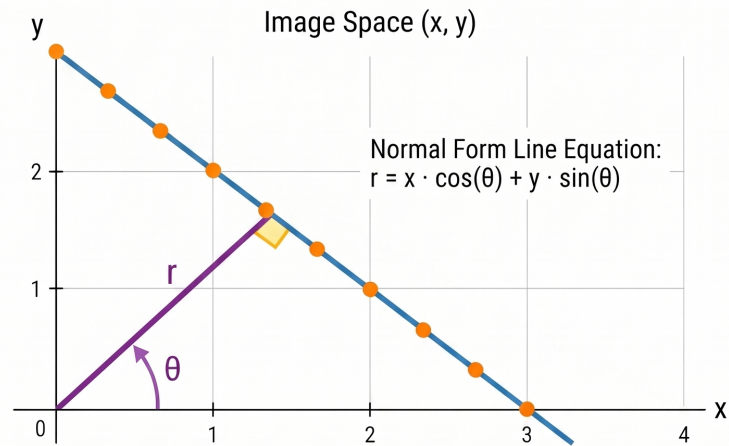
Model	What casts a vote	What it votes for
Hough line detector	An oriented edge pixel	The line (r, θ) it lies on
Generalized Hough transform	An edge, indexed by gradient orientation	The reference center of a known shape, in (x, y, s, θ)
Implicit shape model	A patch matched to a codebook entry	The object center, in a continuous space, then a segmentation

Table 10: The through-line of the section. As the local evidence grows richer — from edge to oriented edge to learned appearance patch — the same voting scheme stretches from a single line to an arbitrary shape to a whole object category.

A Figures: Visual Object Models

The Hough Transform: Detecting Straight Lines

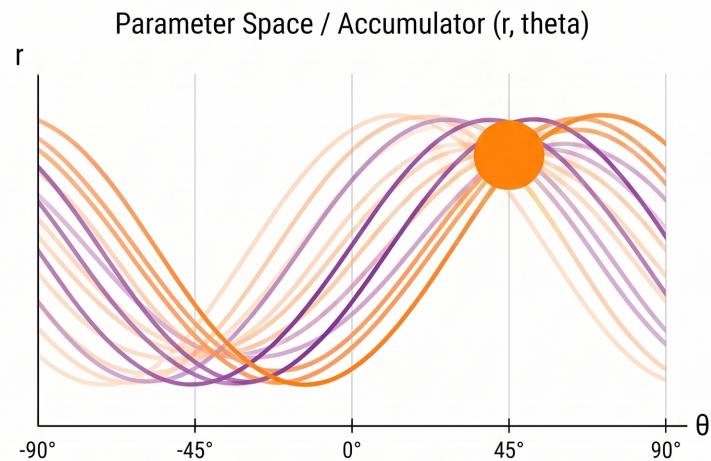
The Hough Transform identifies shapes by mapping image pixels into a parameter space. For straight lines, each pixel votes for all possible lines it could belong to, represented as sinusoidal curves that intersect at a single coordinate (r, θ) corresponding to the detected line.



Definition: A line is defined by its perpendicular distance r and its angle θ from the origin.

Mapping Edge Pixels: Collinear points on a line share a constant (r, θ) relationship used for detection.

Geometry: r is the shortest distance to the line, meeting it at a right-angle foot.



Pixel-to-Curve Transformation: Every image pixel transforms into a unique sinusoidal curve in the (r, θ) plane.

Accumulator Voting: Collinear points result in curves that intersect at one specific coordinate.

Peak Detection: A pronounced maximum in the accumulator triggers the detection of a straight line.

📄 NotebookLM

Figure 13: Hough duality for line detection. Collinear edge points in image space (top) all map to the same point in the (r, θ) accumulator (bottom); each individual point maps to a sinusoid, and the sinusoids intersect where the pixels agree. The pronounced peak triggers the detection.

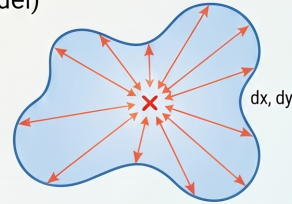
The Generalized Hough Transform: A Two-Stage Process for Shape Detection

Enables detection of arbitrary 2D shapes by mapping edge orientations to displacement vectors through a relational model (R-Table) and probabilistic voting.

Stage 1: Training (Building the Model)

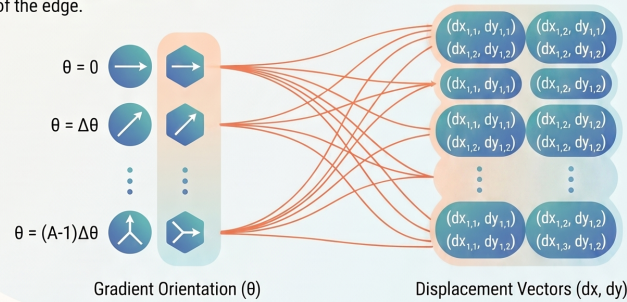
Define Reference Point and Displacement

Select a center point for the arbitrary contour and map displacement vectors from every edge point.



Construct the R-Table

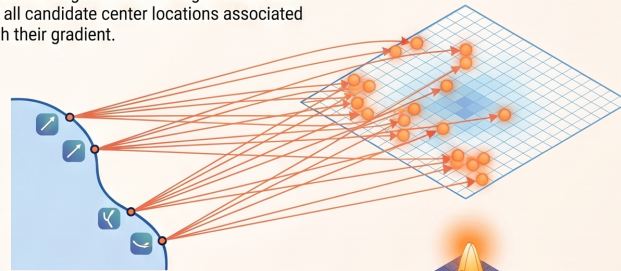
Store displacement vectors (dx, dy) in a table indexed by the local gradient orientation of the edge.



Stage 2: Detection (Object Recognition)

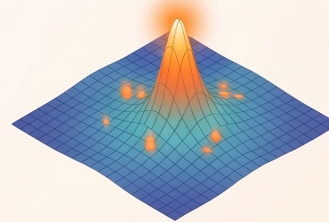
Probabilistic Voting

Observed edges in a test image cast votes for all candidate center locations associated with their gradient.



Accumulator Peak Identification

The true center of the object is identified where the highest density of votes (the pronounced peak) occurs.

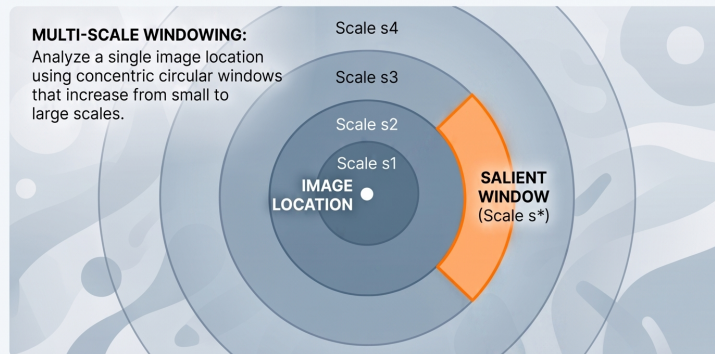


NotebookLM

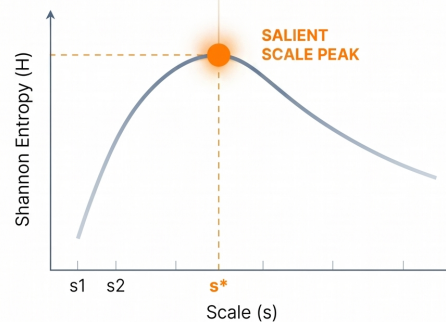
Figure 14: Generalized Hough transform. Each model edge stores a displacement vector to the reference point, organized in the R-table by gradient orientation. Because the same orientation occurs at several places on the shape, one observed edge casts votes for several candidate centers; the true center is where votes accumulate.

Detecting Saliency: The Kadir-Brady Scale-Space Model

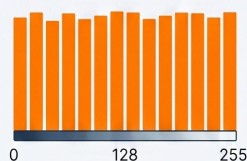
Saliency is a measure of "rareness" or local complexity. This model identifies salient points by analyzing the Shannon Entropy of local image attributes across a range of scales (window sizes) to find where visual information is most distinctive.



THE ENTROPY CURVE $H(s)$:
Calculate local Shannon Entropy at each scale to produce a plot of entropy as a function of scale.
The salient scale is defined as the point where the entropy-versus-scale curve reaches a clear maximum.



FLAT HISTOGRAM
(Evenly Distributed)



HIGH ENTROPY / SALIENCY

A flat local intensity histogram indicates high entropy, signifying a rare and complex image region.

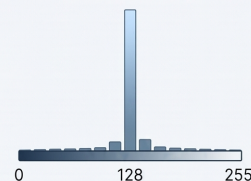
SALIENCY AS LOCAL COMPLEXITY:

High entropy corresponds to a measure of uncertainty, randomness, or average information content.

MEASURE OF SELF-UNSIMILARITY:

Selecting entropy peaks ensures saliency is valid in both the spatial domain and scale space.

PEAKED HISTOGRAM
(Low Uncertainty)



LOW ENTROPY

A peaked histogram results in low entropy, corresponding to low information content and low saliency.

NotebookLM

Figure 15: Scale-space saliency. The same location is analyzed over a range of window sizes; entropy as a function of scale has a peak at the salient scale. A flat local histogram (high entropy) signals a rare, informative region, while the scale-localization weight discounts patches whose statistics barely change across scale.

The Implicit Shape Model (ISM) Pipeline: From Local Patches to Global Detection

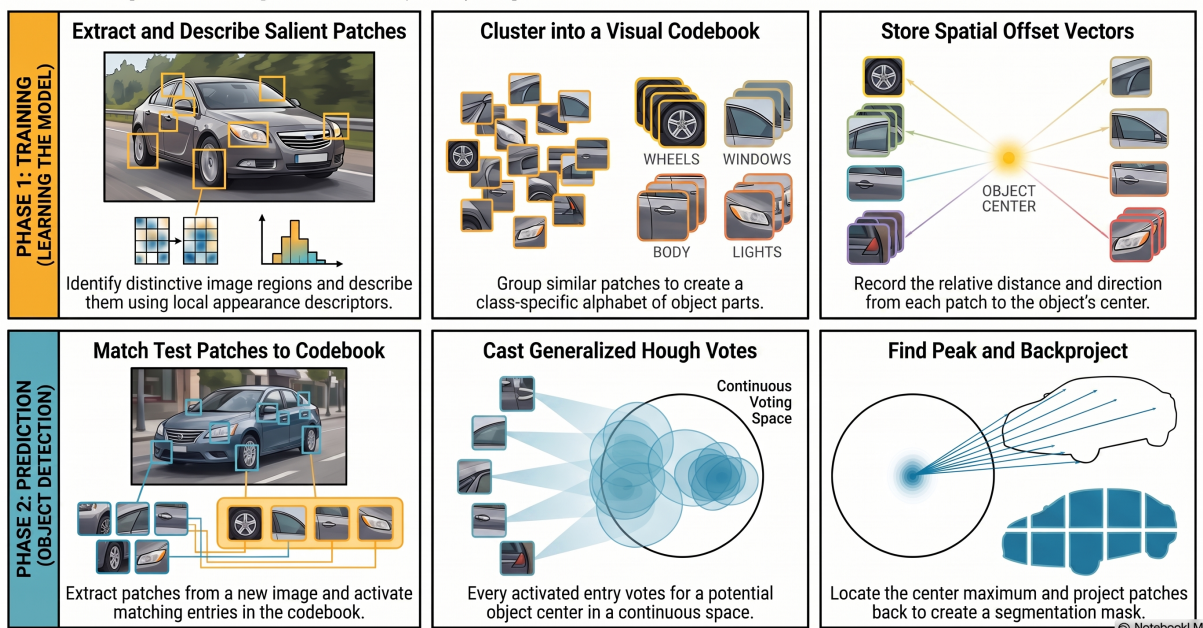


Figure 16: The implicit shape model. Training (top row): salient patches are described, clustered into a codebook, and annotated with offset vectors to the object center. Prediction (bottom row): test patches are matched to the codebook, cast generalized Hough votes for the center, and the peak is back-projected to recover and segment the object.